

Topics in Multiagent Learning

MIT 6.7980 — Fall 2026

Lecture: Tuesdays and Thursdays, 11:00 am–12:30 pm, in room E25-111.

Instructors:

- ▶ Prof. Gabriele Farina, office 45-501F (in the College of Computing building)
✉ gfarina@mit.edu URL: www.mit.edu/~gfarina
- ▶ Prof. Constantinos Daskalakis, office 32-G694 (in the Stata building)
✉ costis@csail.mit.edu URL: people.csail.mit.edu/costis

We are happy to meet with students by appointment.

Teaching assistants:

- ▶ Kat Federova (✉ fedorova@mit.edu). Office hours: TBD.
- ▶ Mingyang Liu (✉ liumy19@mit.edu). Office hours: TBD.
- ▶ Daniel Xia (✉ dxia03@mit.edu). Office hours: TBD.
- ▶ Rui Yao (✉ rayyao@mit.edu). Office hours: TBD.

Grading:

- ▶ Attendance and participation (see below) — 20%.
- ▶ Improving material (see below) — 30%.
- ▶ Project (see below) — 50%.

Attendance: We expect everyone to attend at least 50% of the lectures. The “Attendance and participation” component of grading above reflects (in a binary manner) whether that was met. We will measure attendance using random quizzes during classes.

Coursework: There are no assigned homework sets. Students will contribute to the shared course materials and complete a project, as described below.

GitHub: Main course repository

Prerequisites: Discrete Mathematics and Algorithms at the advanced undergraduate level; mathematical maturity.

Lecture notes: Lecture notes are available as HTML and PDF on the course website. Announcements and administrative materials will be posted on Canvas.

Collaboration policy: We encourage working together on the course materials, projects, and discussion of the ideas. Contributions and project work should reflect *your own* understanding. Acknowledge collaborators and sources, and do not present somebody else’s work as your own.

Acceptable AI use: ¹ Students may use generative AI tools (e.g., LLMs) to support learning, brainstorming, editing, debugging, or generating explanations. We believe that *your human understanding* is still the ultimate goal of education more broadly. We should aim to expand our knowledge, deepen our understanding, and sharpen our thinking. Generative AI might be a great tool for aiding this process, but learning and growth require productive struggle. Don’t let generative AI remove or reduce your productive struggle. We also expect (and require) *full transparency* regarding your use of AI. Always indicate if and how you used generative AI, i.e., which tools you used and during which phases of your work.

¹This policy is inspired *in part* from the [template policy](#) provided by the MIT Teaching + Learning Lab.

Description

This course studies multiagent systems through game theory, optimization, and learning theory. We cover foundational topics such as Nash equilibria, regret minimization, learning dynamics, and extensive-form games.

We also explore modern topics: multiagent deep reinforcement learning; information and mechanism design; team games and hidden-role games; alignment; high-dimensional and kernelized learning; nonconvex games; calibration; and the complexity of finding equilibria. Applications and open research questions connect the theory to multiagent AI.

Improving Material

We would like to make the lecture notes available to as many people as possible. You can now read them in a browser, follow links between sections and references, and move between the notes and their source. We would like everyone's help to make this a useful resource for learners around the world.

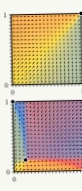
MIT 6.7980 - Fall 2026

Topics in Multiagent Learning

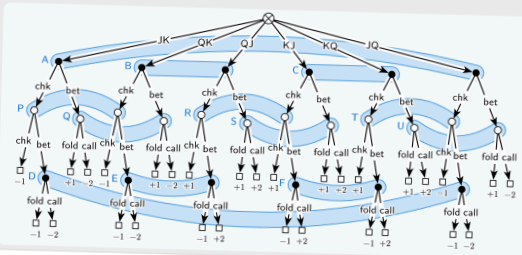
This course studies multiagent systems through game theory, optimization, and learning theory. We cover foundational topics such as equilibria, regret minimization, and learning dynamics in games.

We also explore modern topics, including multiagent deep reinforcement learning, calibration and alignment, and hidden-role games, connecting theory to applications and open research questions in multiagent AI.

[Schedule & notes](#) [Syllabus \(PDF\)](#) [Fog of War Chess challenge](#)



Example L7.1. As a running example, we will illustrate the representation by analyzing the game tree of a simplified two-player variant of poker (perhaps the archetype of imperfect-information extensive-form games), known as *Kuhn poker* [Kuh50]. As noted by Kuhn himself, even the previous small game already captures central aspects of deceptive behavior such as *bluffing* and *underbidding*.



MIT 6.7980 - Fall 2026

Topics in Multiagent Learning

Concise Introduction and Lecture Notes

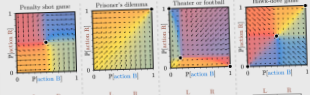
LECTURES

1. Setting and equilibria: the Nash equilibrium
2. Regret minimization, Nash, Brown, and Spence
3. Properties and relaxations of the Nash equilibrium
4. Foundations of learning in games
5. Learning algorithms (I)
6. Backward induction
7. Foundations of extensive form games
8. Learning in extensive form games
9. FPGD: foundations of Nash equilibria

TOPICS

- 1.1.1. Nash equilibrium
- 1.1.2. Nash equilibrium
- 1.1.3. Nash equilibrium
- 1.1.4. Nash equilibrium
- 1.1.5. Nash equilibrium
- 1.1.6. Nash equilibrium
- 1.1.7. Nash equilibrium
- 1.1.8. Nash equilibrium
- 1.1.9. Nash equilibrium
- 1.1.10. Nash equilibrium
- 1.1.11. Nash equilibrium
- 1.1.12. Nash equilibrium
- 1.1.13. Nash equilibrium
- 1.1.14. Nash equilibrium
- 1.1.15. Nash equilibrium
- 1.1.16. Nash equilibrium
- 1.1.17. Nash equilibrium
- 1.1.18. Nash equilibrium
- 1.1.19. Nash equilibrium
- 1.1.20. Nash equilibrium
- 1.1.21. Nash equilibrium
- 1.1.22. Nash equilibrium
- 1.1.23. Nash equilibrium
- 1.1.24. Nash equilibrium
- 1.1.25. Nash equilibrium
- 1.1.26. Nash equilibrium
- 1.1.27. Nash equilibrium
- 1.1.28. Nash equilibrium
- 1.1.29. Nash equilibrium
- 1.1.30. Nash equilibrium
- 1.1.31. Nash equilibrium
- 1.1.32. Nash equilibrium
- 1.1.33. Nash equilibrium
- 1.1.34. Nash equilibrium
- 1.1.35. Nash equilibrium
- 1.1.36. Nash equilibrium
- 1.1.37. Nash equilibrium
- 1.1.38. Nash equilibrium
- 1.1.39. Nash equilibrium
- 1.1.40. Nash equilibrium
- 1.1.41. Nash equilibrium
- 1.1.42. Nash equilibrium
- 1.1.43. Nash equilibrium
- 1.1.44. Nash equilibrium
- 1.1.45. Nash equilibrium
- 1.1.46. Nash equilibrium
- 1.1.47. Nash equilibrium
- 1.1.48. Nash equilibrium
- 1.1.49. Nash equilibrium
- 1.1.50. Nash equilibrium
- 1.1.51. Nash equilibrium
- 1.1.52. Nash equilibrium
- 1.1.53. Nash equilibrium
- 1.1.54. Nash equilibrium
- 1.1.55. Nash equilibrium
- 1.1.56. Nash equilibrium
- 1.1.57. Nash equilibrium
- 1.1.58. Nash equilibrium
- 1.1.59. Nash equilibrium
- 1.1.60. Nash equilibrium
- 1.1.61. Nash equilibrium
- 1.1.62. Nash equilibrium
- 1.1.63. Nash equilibrium
- 1.1.64. Nash equilibrium
- 1.1.65. Nash equilibrium
- 1.1.66. Nash equilibrium
- 1.1.67. Nash equilibrium
- 1.1.68. Nash equilibrium
- 1.1.69. Nash equilibrium
- 1.1.70. Nash equilibrium
- 1.1.71. Nash equilibrium
- 1.1.72. Nash equilibrium
- 1.1.73. Nash equilibrium
- 1.1.74. Nash equilibrium
- 1.1.75. Nash equilibrium
- 1.1.76. Nash equilibrium
- 1.1.77. Nash equilibrium
- 1.1.78. Nash equilibrium
- 1.1.79. Nash equilibrium
- 1.1.80. Nash equilibrium
- 1.1.81. Nash equilibrium
- 1.1.82. Nash equilibrium
- 1.1.83. Nash equilibrium
- 1.1.84. Nash equilibrium
- 1.1.85. Nash equilibrium
- 1.1.86. Nash equilibrium
- 1.1.87. Nash equilibrium
- 1.1.88. Nash equilibrium
- 1.1.89. Nash equilibrium
- 1.1.90. Nash equilibrium
- 1.1.91. Nash equilibrium
- 1.1.92. Nash equilibrium
- 1.1.93. Nash equilibrium
- 1.1.94. Nash equilibrium
- 1.1.95. Nash equilibrium
- 1.1.96. Nash equilibrium
- 1.1.97. Nash equilibrium
- 1.1.98. Nash equilibrium
- 1.1.99. Nash equilibrium
- 1.1.100. Nash equilibrium

Example L7.2. The plots below visualize the displacement $g(x_1, x_2) = (x_1, x_2)$ induced by the Nash improvement function for four games, where payoff matrices are noted below each plot, after projecting away the probability of the first action of each player (and keeping around only the probability of the second action, which is sufficient to uniquely recover the strategy of the player since each player only has two actions). The black dots denote the fixed points of the Nash improvement function. These correspond exactly to the Nash equilibria of the game, as we make formal below.



The background of the plots highlights the angle of displacement induced by the Nash improvement function, according to the gradient wheel shown below.



[If the color scheme seems arbitrary, as a small spoiler it will play a fundamental role in the proof of the computational complexity of Nash equilibria, which we will discuss later on in this course. In particular, the three regions of the coloring scheme will be key in defining an important combinatorial construction called *Spencer coloring*.]

3.3.1.2 Nash equilibrium in two-player general-sum games

In the general two-player case, when referred to as two-player general-sum games, many of the nice properties of the zero-sum case are lost.

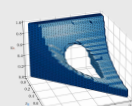
Topological properties Perhaps the most striking is that not only the set of Nash equilibria is no longer guaranteed to be convex, but it is not even guaranteed to be nonempty. We show this phenomenon with the next example.

Remark 3.3.5 (Complex topology of Nash equilibria, Kohlberg-Mertens game [KM86]).

Beyond two-player zero-sum games, the set of Nash equilibria in a game can be quite complex. For one, it is not at all guaranteed that the set is convex. Even more, the set might be topologically complex, e.g., exhibiting holes. This phenomenon was already observed by Kohlberg and Mertens [KM86], who considered the two-player three-action game

	A	B	C
A	1, 1	0, 1	-1, 1
B	-1, 0	0, 0	-1, 0
C	1, -1	0, -1	-1, -2

The figure on the right, similar to the one in [KM86], shows the projection of the set of all 0.27 approximate Nash equilibria of the game, i.e., all strategy profiles such that no player has a unilateral deviation that increases their utility by more than 0.27.



Computation in two-player general-sum games. computation of Nash equilibria is not a linear program. However, it is a linear complementarity problem (LCP), a more general class of problems than linear feasibility programs, and which are written in the form

$$\text{find } x, w \in \mathbb{R}^d \quad \text{s.t.} \quad w = Mx + g, \quad x_i w_i = 0, \quad x^* w = 0.$$

The Lemke-Howson algorithm is a well-known algorithm to solve LCPs, and it can be used to find Nash equilibria in two-player general-sum games. However, the algorithm is not relevant to this course.

We will divide the class into groups, each focusing on a different part of the material. Using the class GitHub repository, each group can open issues to identify improvements and submit pull requests to implement them. We will improve the material together, reviewing and building on one another's contributions.

Contributions can include clarifying explanations and proofs, fixing errors, adding examples and homework-style exercises for future readers, and polishing figures, organization, and presentation. If anyone is brave enough, we would also love interactive components that let readers experiment with the ideas.

On the bright side, there is no homework! :-) Improving the shared material accounts for 30% of the course grade.

Project

Projects may be completed individually or in groups of 2-5 students and will include a presentation. We will offer three project directions:

Fog of War Chess Challenge. Build and evaluate an agent that plays with partial information. Explore how it uses observations, reasons about uncertainty, and chooses strategic actions. Each bot sandbox is allocated two CPU cores and 4 GiB of memory. A dedicated document will describe the challenge, including the rules, starter code, and how to access the arena.

The screenshot shows the MIT 6.7980 Fall 2026 "Fog of War Challenge" interface. It features a central chessboard with a fog of war effect, where some squares are obscured by a dark overlay. The interface includes a leaderboard on the left, a match details panel on the right, and a replay controls bar at the bottom. The match details panel shows a result of 1/2 - 1/2, a threefold repetition, and a time limit of 3:00 + 2s / move. The field of vision is 23 / 64 squares, with 41 squares hidden from White. The move history shows 134 plies. The replay controls bar shows a progress bar at 52 / 134 and a 4x speed setting.

MIT 6.7980 · Fall 2026

Fog of War Challenge

Submit a bot

Arena Leaderboard Games Rules

Example arena

Leaderboard

#	BOT / TEAM	ELO
1	Example White Supplied example engine	1500
1	Example Black Supplied example engine	1500

One supplied game. Example engines only; student rankings will appear when the course arena opens.

Explore rankings → 2 bots

REPLAY / EXAMPLE 001

P1 POV P2 POV True state

2:39

Example Black Player 2 · Black

Example White Player 1 · White

2:04

52 / 134

4x

Match details

FINAL

RESULT

1/2 - 1/2

Threefold repetition

3:00 + 2s / move

FIELD OF VISION

23 / 64 squares

41 squares hidden from White.

Move history 134 plies

Move notation is hidden in POV mode to preserve the player's information. Step through the board to inspect observations.

Modeling questions. Formulate a multiagent problem by specifying the players, objectives, information, and available actions. Study how modeling choices affect the resulting strategic behavior. We will provide a separate document with possible modeling questions and leads to explore.

Theory questions. Investigate a mathematical question about equilibria, learning dynamics, or computational complexity. Develop rigorous proofs, bounds, or counterexamples that clarify the behavior of multiagent systems. We will provide a separate document with possible theory questions and leads to explore.

The project is the central component of the course and accounts for 50% of the final grade. We will therefore be “robust” in our grading: we will look carefully at the depth of your understanding, the quality and substance of your work, and how clearly you explain your results.

Tentative Schedule

0	Sep 10 th	Course Overview We will discuss the syllabus, projects, administrative details, and an overview of the topics covered.
PART I: FOUNDATIONS		
1	Sep 15 th	Setting and equilibria: the Nash equilibrium Nash's existence theorem and its connection to fixed-point theorems.
2	Sep 17 th	Brouwer and Sperner Sperner's lemma, Brouwer's theorem, and combinatorial proofs of equilibrium existence.
3	Sep 22 nd	Properties of Nash equilibrium Topological and computational properties. Zero-sum games and linear programming. Correlated and coarse correlated equilibria.
4	Sep 24 th	Learning in games: Foundations Regret and hindsight rationality. Regret minimization and its relationships with equilibrium concepts.
5	Sep 29 th	Learning in games: Algorithms General principles for learning algorithms. Follow-the-leader, regret matching, multiplicative weights, and online mirror descent.
6	Oct 1 st	Learning with bandit feedback Partial feedback and exploration. From multiplicative weights to Exp3; regret guarantees.
7	Oct 6 th	Modeling extensive-form games Perfect and imperfect information. Kuhn's theorem. Normal-form and sequence-form strategies.
8	Oct 8 th	Learning in extensive-form games No-regret learning, counterfactual utilities, and counterfactual regret minimization (CFR).
	Oct 13 th	No class MIT follows a Monday schedule.
9	Oct 15 th	Multiagent deep RL Reinforcement learning in games. Self-play and deep learning methods for perfect-information games.
10	Oct 20 th	Taking stock We will discuss big open questions in the field and possible project ideas.
PART II: INFORMATION, COMMUNICATION, ALIGNMENT		
11	Oct 22 nd	Information and mechanism design Designing information and incentives in strategic interactions.

12	Oct 27 th	Team games and hidden-role games Coordination in teams and games with hidden roles.
13	Oct 29 th	Alignment (part I) Reinforcement learning from human feedback (RLHF) and alignment.
14	Nov 3 rd	Alignment (part II) Regularized RLHF and direct preference optimization (DPO).
PART III: ADVANCED LEARNING		
15	Nov 5 th	Forecasting and calibration Calibrated prediction and its connections to learning in games.
16	Nov 10 th	High-dimensional games Combinatorial games, vertex MWU, and the 0/1-polyhedral kernel. Efficient kernels and sequence-form learning.
17	Nov 12 th	Nonconvex games Nonconvexity, learning dynamics, and local equilibrium concepts.
PART IV: COMPUTATIONAL COMPLEXITY		
18	Nov 17 th	Total search and TFNP Total search and polynomially verifiable witnesses. Succinct End-of-Line reductions and the PPAD complexity class.
19	Nov 19 th	PPAD-hardness of Nash equilibrium Reductions and the computational hardness of finding Nash equilibria.
PROJECT WORK AND PRESENTATIONS		
	Nov 24 th BREAK	No class Project break
	Nov 26 th NO CLASS	No class Thanksgiving holiday.
20	Dec 1 st PROJECT	Project presentations Show us your cool work!
21	Dec 3 rd PROJECT	Project presentations Show us your cool work!
22	Dec 8 th PROJECT	Project presentations Show us your cool work!
23	Dec 10 th PROJECT	Project presentations Show us your cool work!

Supplementary lecture: Fast computation via the minimax theorem. Based on Farina's SIGecom Exchanges article. Defense oracles, the ellipsoid construction, logarithmic dependence on inverse accuracy, and Phi-equilibrium computation via fixed points of deviations.